Instituto Tecnológico Superior de Xalapa

INSTITUTO TECNOLÓGICO SUPERIOR DE XALAPA

ING. EN SISTEMAS COMPUTACIONALES

3° Semestre | Grupo: B

MÉTODOS NUMÉRICOS

VAZQUES HERRERA VICTOR HUGO

UNIDAD 3
TAREA 2

PRESENTA

257O00081
Brayan Mejia Mora

18 DE OCTUBRE DEL 2026

Práctica: Métodos Iterativos

1. Estructura de las clases

1.1 Paquete Unidad3y4.Jacobi

Clase DefMatriz

Clase utilitaria que centraliza los datos del sistema Ax = b. No tiene estado; solo expone métodos estáticos que devuelven la matriz y los vectores.

```
==========================================================================
DefMatriz
├── + obtenerMatrizA() : double[][]
├── + obtenerVectorB() : double[]
└── + obtenerVectorX0() : double[]
==========================================================================
```

Clase Jacobi

Contiene la implementación del método iterativo de Jacobi y la función auxiliar de error (reutilizada por Gauss-Seidel).

```
==========================================================================
Jacobi
├── + resolver(A, b, x0, tol, maxIter) : double[]
└── + calcularError(xAnt, xNuevo) : double[]
==========================================================================
```

Clase Main (Jacobi)

Punto de entrada: carga datos desde DefMatriz, invoca Jacobi.resolver e imprime el vector solución.

```
==========================================================================
Main
└── + main(String[] args) : void
==========================================================================
```

1.2 Paquete Unidad3y4.GaussSeidel

Clase GaussSeidel

Implementación del método de Gauss-Seidel. Depende de Unidad3y4.Jacobi.Jacobi únicamente para reutilizar calcularError.

```
================================================================================
GaussSeidel
└── + resolver(A, b, xInicial, tol, maxIter) : double[]
================================================================================
```

Clase Main (Gauss-Seidel)

Punto de entrada: recicla A y b desde DefMatriz (del paquete Jacobi), construye un vector inicial en ceros e invoca GaussSeidel.resolver.

```
================================================================================
Main
└── + main(String[] args) : void
================================================================================
```

2. Fragmentos clave del código

2.1 Núcleo iterativo de Jacobi (Jacobi.java)

```
================================================================================
for (int i = 0; i < n; i++) {
    double suma = 0.0;

    for (int j = 0; j < n; j++) {
        if (j != i) {                    // excluye la diagonal
            suma += A[i][j] * x[j];      // usa el vector VIEJO (x)
        }
    }

    xNuevo[i] = (b[i] - suma) / A[i][i];
}

if (calcularError(x, xNuevo) < tol) return xNuevo;
x = xNuevo.clone();
================================================================================
```

Característica clave: se escribe en xNuevo y se lee de x, por lo que todos los componentes se actualizan simultáneamente con los valores de la iteración anterior.

2.2 Núcleo iterativo de Gauss-Seidel (GaussSeidel.java)

```
================================================================================
for (int i = 0; i < n; i++) {
    double suma = b[i];

    for (int j = 0; j < n; j++) {
        if (j != i) {
            suma -= A[i][j] * x[j];      // usa valores RECIÉN calculados
        }
    }

    x[i] = suma / A[i][i];               // actualización directa
}

if (Jacobi.calcularError(xAnt, x) < tol) return x;
================================================================================
```

Característica clave: un solo vector x se actualiza en directo; al llegar a la ecuación i, los componentes x[0..i-1] ya son de la iteración actual.

2.3 Criterio de parada compartido (Jacobi.calcularError)

```
================================================================================
public static double calcularError(double[] xAnt, double[] xNuevo) {
    double max = 0.0;

    for (int i = 0; i < xAnt.length; i++) {
        max = Math.max(max, Math.abs(xNuevo[i] - xAnt[i]));
    }

    return max;   // norma infinito de la diferencia
}
================================================================================
```

3. Repositorio público

Enlace: https://github.com/brianlewyn/numerical-methods-tec-3th-semester/tree/main/Unidad3y4

```
================================================================================
/
├── Unidad3y4/
│   ├── Jacobi/
│   │   ├── DefMatriz.java
│   │   ├── Jacobi.java
│   │   ├── Main.java
│   │   └── README.md
│   └── GaussSeidel/
│       ├── GaussSeidel.java
│       ├── Main.java
│       └── README.md
├── Tarea 1 Unidad 3 (Métodos Numéricos).pdf
└── Tarea 2 Unidad 3 (Métodos Numéricos).pdf
================================================================================
```

4. Conclusión técnica: comparación de iteraciones

Para cuantificar la diferencia de convergencia se instrumentó el método Jacobi.resolver() y GaussSeidel.resolver() con un contador de iteraciones, ejecutando ambos sobre el mismo sistema definido en DefMatriz:

```
==========================================================================
A = | 5  1  1 |      b = | 7 |      x0 = [0, 0, 0]      tol = 1e-4
    | 1  4  1 |          | 6 |
    | 1  2  5 |          | 8 |
==========================================================================
```

Resultados obtenidos

Método	**Iteraciones requeridas**	**Solución aproximada**
Jacobi	20	[1.00000, 1.00000, 0.99996]
Gauss-Seidel	6	[0.99999, 1.00000, 1.00000]

Análisis

- **Menor número de iteraciones en Gauss-Seidel**. Gauss-Seidel necesitó 6 iteraciones frente a las 20 de Jacobi para alcanzar la misma tolerancia (0.0001). Esto representa una reducción de aproximadamente 70% (100 - 100/20*6) en el número de iteraciones.

- **Causa del mejor desempeño**. Al usar inmediatamente los valores recién calculados, Gauss-Seidel propaga la información dentro de la misma iteración. En un sistema diagonalmente dominante como el de prueba (cada |a_ii| supera la suma de los |a_ij| de su fila: 5>2, 4>2, 5>3), esta propagación acelera notablemente la contracción del error.

- **Costo por iteración**. Jacobi requiere un vector auxiliar (xNuevo) y es naturalmente paralelizable, ya que todas las componentes se calculan de forma independiente. Gauss-Seidel es inherentemente secuencial dentro de cada iteración, pero compensa con menos iteraciones totales.

- **Misma precisión final**. Ambos métodos convergen al mismo vector solución **[1, 1, 1]**; la diferencia está únicamente en la velocidad de convergencia, no en el resultado.

- **Conclusión práctica**. Para sistemas diagonalmente dominantes de tamaño moderado, Gauss-Seidel es preferible por su menor número de iteraciones y menor uso de memoria. Jacobi conserva ventaja en contextos de cómputo paralelo o cuando la matriz es muy dispersa y se busca simplicidad de implementación.

Nota: Los conteos exactos (20 vs 6) dependen de la tolerancia y del redondeo de punto flotante; pueden variar ligeramente entre plataformas, pero la relación Gauss-Seidel < Jacobi en iteraciones se mantiene mientras el sistema sea diagonalmente dominante.